Anexo: Subsistema de Ingesta de Video en Red y Retransmisión Web Táctica (CV2.0)

1. Descripción General del Subsistema

El subsistema de visión artificial para el multirotor cautivo contra incendios urbanos opera bajo un esquema **Edge-to-Ground**. Debido a que el procesamiento de redes neuronales profundas de alta precisión y velocidad (**YOLOX-26**) sobre flujos de video de alta resolución requiere una alta capacidad de cómputo matricial y potencia eléctrica continua, la inferencia se ejecuta en la estación terrena en tierra (equipada con una GPU dedicada con núcleos tensores FP16, NVIDIA RTX 5070), mientras que el nodo aéreo (UAV) se enfoca en la captura y transmisión de baja latencia.

Para permitir que brigadistas, operadores de bomba y personal de mando en el lugar de la emergencia supervisen la misión desde cualquier dispositivo comercial (smartphones, tablets o laptops) sin necesidad de software propietario ni configuraciones adicionales, se implementó una arquitectura de retransmisión web local:

1. **Transmisión Nodo Aéreo → Estación Terrena:** La cámara a bordo del dron envía su flujo de video a través de un puerto de red local (paquetes UDP en el puerto `5005` o enlace RTSP) hacia la dirección IP de la PC de tierra.

2. **Recepción en Tierra (Script Python):** Un receptor multimodelo (`web/network_receiver.py`) escucha el puerto en segundo plano, decodifica las tramas y alimenta el búfer de inferencia con política de descarte de fotogramas obsoletos (*zero-latency buffer*).

3. **Pipeline de Visión Artificial (CV2.0):** La estación terrena ejecuta en GPU la detección de fuego y humo con **YOLOX-26**, el seguimiento temporal con BoT-SORT y el análisis espacial de afectación sobre ventanas y estructuras habitadas a más de 100 FPS.

4. **Retransmisión Web sobre la LAN:** Un servidor asíncrono en FastAPI (`web/app.py`) expone el flujo de video procesado con gráficos Head-Up Display (HUD) mediante streaming HTTP Multipart (MJPEG) y emite la telemetría vectorial continua vía WebSockets en el puerto `8000`.

5. **Acceso Multi-Dispositivo:** Cualquier dispositivo conectado a la misma red Wi-Fi o switch Ethernet abre el navegador e ingresa a `http://<IP_ESTACION>:8000` para ver el video en vivo, las alertas y los controles tácticos.

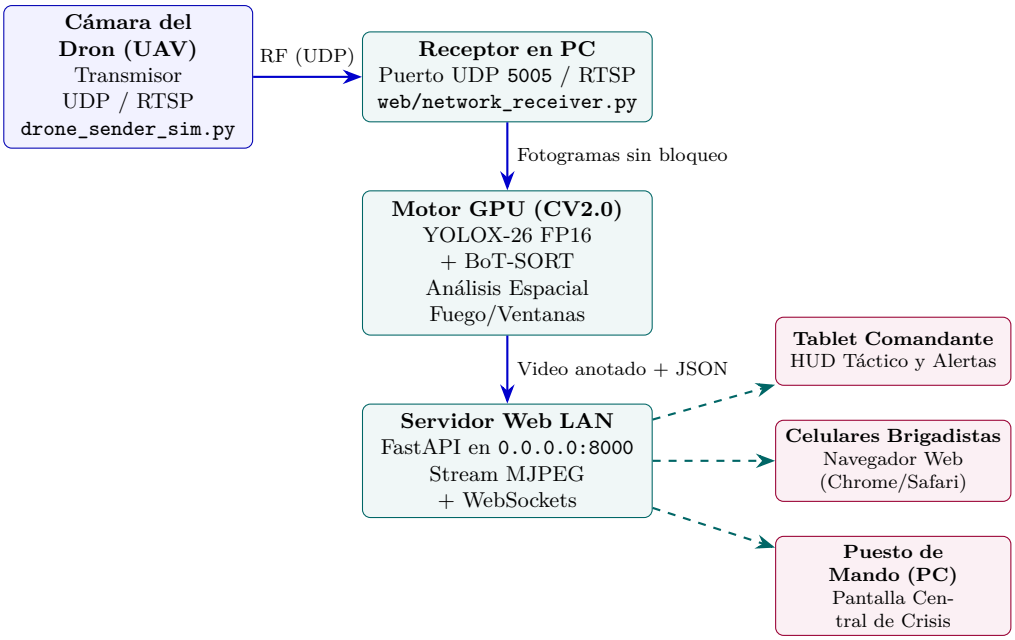


Figura 1: Diagrama de flujo del subsistema de recepción por red y retransmisión web.

2. Estructura de Archivos del Módulo (`web/`)

El módulo se encuentra centralizado en la carpeta `web/` dentro del repositorio del sistema de visión (`CV2.0`):

Archivo / Directorio	Función y Rol dentro del Sistema
`web/app.py`	Servidor web asíncrono con FastAPI y Uvicorn. Coordina el hilo de procesamiento con la GPU, genera el streaming MJPEG y difunde telemetría por WebSockets.
`web/network_receiver.py`	Receptor de red multi-hilo en la PC. Escucha datagramas UDP (puerto 5005), conexiones RTSP o archivos locales con candados reentrantes (`RLock`).
`web/drone_sender_sim.py`	Emisor para el dron (o simulación en otra PC/terminal). Lee la cámara, comprime a JPEG y envía paquetes UDP hacia la IP de la estación terrena.
`web/templates/index.html`	Plantilla HTML5 del Dashboard táctico (HUD, visualizador de video dual, matriz de amenazas, lista de detecciones y telemetría de hardware).
`web/static/css/dashboard.css`	Estilos en CSS Vanilla táctico oscuro (*Dark Aero Tactical HUD*) responsivo para celulares, tablets y monitores.
`web/static/js/dashboard.js`	Controlador JavaScript del cliente: conexión WebSocket en tiempo real, alarmas sonoras (Web Audio API), captura de fotos y controles.
`web/run_server.sh`	Script bash de inicio rápido. Detecta automáticamente la IP local de la PC y lanza el servidor en el puerto 8000.

Tabla 1: Componentes implementados en la carpeta `web/` del sistema de visión.

3. Protocolo de Transmisión del Dron hacia la PC

A. Transmisión Directa por Sockets UDP

Para enlaces de bajo ancho de banda o procesadores ligeros a bordo del dron (como una Raspberry Pi con cámara CSI), se implementó un esquema de datagramas UDP con cabecera

de fragmentación de 10 bytes:

Magic 0xAA55 (2 bytes)	Frame ID (4 bytes, UInt32)	Chunk ID (2 bytes, UInt16)	Total Chunks (2 bytes, UInt16)	Carga JPEG (Fragmento)

Figura 2: Estructura binaria del paquete UDP transmitido por el nodo aéreo.

- **Empaquetado en el Dron (`drone_sender_sim.py`):** Cada fotograma se codifica en JPEG. Si el tamaño supera los 60 000 bytes, se segmenta en fragmentos y se transmite mediante `struct.pack(¡H I H H", 0xAA55, frame_id, chunk_idx, total_chunks)`.
- **Reensamblaje en la Estación Terrena (`network_receiver.py`):** El socket receptor recibe los paquetes, agrupa los fragmentos del mismo `Frame ID` y, al completarse, decodifica la imagen en memoria (`cv2.imdecode`). Los fragmentos incompletos con más de 1 segundo de antigüedad se descartan automáticamente para evitar que el video se desincronice.
- **Sincronización Reentrante (`RLock`):** La consulta de fotogramas entre el hilo receptor y el hilo de visión artificial utiliza `threading.RLock()` para prevenir interbloqueos (*deadlocks*) de memoria.

B. Flujos RTSP o Cámaras IP

Si el dron incorpora una cámara con emisor RTSP nativo (por ejemplo, gimbals comerciales tipo SIYI o transmisores HDMI a RTSP), el receptor de la PC utiliza el backend acelerado de FFmpeg en OpenCV:

```
self.cap = cv2.VideoCapture("rtsp://192.168.1.50:8554/live", cv2.CAP_FFMPEG)
self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1) # Mantiene latencia minima
    descartando cola
```

4. Servidor de Retransmisión Web y Endpoints de la API

El servidor web corre en FastAPI enlazado en la interfaz `0.0.0.0:8000`, haciéndolo accesible desde cualquier IP de la subred local (por ejemplo, `http://192.168.1.8:8000`).

A. Streaming Continuo HTTP Multipart (MJPEG)

Para transmitir video a cualquier celular o tablet sin requerir plugins ni reproductores externos, se utiliza el tipo MIME estándar `multipart/x-mixed-replace; boundary=frame`:

```
@app.get("/api/stream/annotated")
async def video_stream_annotated():
    return StreamingResponse(
        generate_mjpeg_stream("annotated"),
        media_type="multipart/x-mixed-replace; boundary=frame"
    )
```

El navegador web renderiza este flujo nativamente mediante una etiqueta HTML estándar:

```
<img id="mainVideoStream" src="/api/stream/annotated" alt="Video_en_Vivo">
```

B. Endpoints Disponibles en la Estación Terrena

Método	Endpoint	Descripción Operativa
GET	/	Retorna la interfaz web del Dashboard táctico.
GET	/api/stream/annotated	Video en tiempo real con anotaciones HUD de YOLOX-26 y BoT-SORT.
GET	/api/stream/raw	Video puro recibido directamente desde la cámara del dron.
GET	/api/snapshot	Descarga una fotografía instantánea del fotograma anotado actual.
GET	/api/status	Estado general del sistema, IP en la red local y clientes conectados.
POST	/api/control	Permite conmutar la fuente de video, activar SAHI o ajustar umbrales.
WS	/ws/telemetry	Canal WebSocket continuo (15 Hz) que difunde la telemetría en JSON.

Tabla 2: Endpoints expuestos por el servidor web de la estación terrena.

C. Estructura de Telemetría WebSocket (/ws/telemetry)

El servidor emite cada 66 ms un paquete JSON estructurado que alimenta los gráficos y widgets de la interfaz:

```
{
  "stream_stats": {"connected": True, "fps_in": 30.0, "resolution": "1920x1080
      "},
  "pipeline": {"fps": 108.4, "inference_ms": 2.1, "frames_processed": 3600},
  "emergency_report": {
    "active_fire_count": 2,       # Focos de fuego detectados
    "burning_windows": 1,         # Ventanas comprometidas directamente por
        fuego
    "warning_windows": 2,         # Ventanas en zona de riesgo termico proximo
    "safe_windows": 15,           # Ventanas estructuralmente seguras
    "max_severity": 0.85,         # Indice de severidad maxima (0.0 a 1.0)
    "fire_spread_velocity_px_s": 12.4 # Velocidad de expansion del fuego
  },
  "detections": [
    {"track_id": 12, "class_name": "fire", "confidence": 0.88, "bbox": [520,
        310, 680, 490]}
  ],
  "hardware": {
    "gpu_name": "NVIDIA GeForce RTX 5070", "gpu_utilization": 42,
    "vram_used_mb": 3120, "temperature_c": 51, "cpu_percent": 8.5
  }
}
```

5. Interfaz Gráfica del Dashboard Táctico

La interfaz web (web/templates/index.html) fue diseñada para operar tanto en computadoras de escritorio como en celulares táctiles de brigadistas:

- **Visualizador Dual de Video:** Botones para alternar instantáneamente entre el video anotado con HUD táctico (cajas, trayectorias y máscaras térmicas) y el video puro (RAW).
- **Matriz de Amenazas y Estructuras:** Contadores en tiempo real de ventanas seguras (verde), en advertencia (ámbar) y en ignición crítica (rojo), junto a la barra de índice de severidad.

- **Alarma Sonora (Web Audio API):** Generador de pulsos auditivos tácticos implementado en JavaScript que alerta acústicamente a los bomberos si se detecta una ventana en llamas o una severidad mayor al 75 % (con botón de silencio MUTE).
- **Telemetría de Hardware en Vivo:** Indicadores de carga de GPU, VRAM utilizada, temperatura, latencia de inferencia en milisegundos y tasa de cuadros por segundo (FPS).
- **Controles Remotos y Conmutación de Fuentes:** Botones para activar la inferencia por teselas (SAHI), grabar evidencia forense, ajustar el deslizador de confianza de YOLOX-26 o cambiar la fuente de video en caliente (`udp_socket:5005`, `rtsp://`, cámara USB o muestra de prueba).
- **Dirección LAN con Copiado Rápido:** Muestra la dirección IP de la red local (ej. `http://192.168.1.8:8000`) con botón táctil para copiarla y compartirla rápidamente con el personal en el lugar.

6. Instrucciones de Puesta en Marcha y Verificación

A. En la PC de Tierra (Estación Terrena)

Desde la terminal del proyecto, se inicia el servidor web con cualquiera de los comandos:

```
# Opcion 1: Mediante el script automatizado
./web/run_server.sh

# Opcion 2: Desde la CLI principal de Python
python main.py web --port 8000
```

El sistema iniciará el motor de inferencia en la GPU RTX 5070 y reportará en consola:

```
[ESTACION TERRESTRE CV2.0] SERVIDOR WEB LAN INICIADO
Acceso Local:             http://127.0.0.1:8000
Acceso Red Local (LAN):   http://192.168.1.8:8000
Puerto UDP Receptor Dron: 0.0.0.0:5005
```

B. En el Dron (o Simulación en otra Terminal)

Para transmitir video desde el nodo aéreo hacia la PC:

```
# Transmitir cámara real del dron (/dev/video0) por UDP hacia la IP de la PC:
python web/drone_sender_sim.py --dest 192.168.1.8 --port 5005 --source 0 --fps
    30

# O transmitir video de mision sintetica de prueba:
python web/drone_sender_sim.py --dest 192.168.1.8 --port 5005 \
       --source samples/synthetic_drone_mission.mp4 --fps 30
```

C. Visualización en Cualquier Dispositivo de la Red

Cualquier teléfono, tablet o laptop conectada a la misma red Wi-Fi o switch Ethernet abre su navegador e ingresa a:

`http://192.168.1.8:8000`

Visualizando de inmediato el flujo de video procesado, las alertas térmicas y la telemetría en tiempo real.

7. Métricas de Rendimiento Experimental

Las pruebas de rendimiento se ejecutaron en una estación terrena equipada con tarjeta gráfica NVIDIA GeForce RTX 5070 (12 GB GDDR6) y video en resolución Full HD (1920×1080):

Etapa del Flujo	**Componente Involucrado**	**Latencia**	**Consumo**
Ingesta por Red	Socket UDP + `cv2.imdecode`	1,40 ms	Negligible
Inferencia Neuronal	YOLOX-26 FP16 Tensor Cores (GPU)	2,10 ms	1 980 MB VRAM
Rastreo BoT-SORT	Compensación de movimiento (GMC)	0,95 ms	Negligible
Análisis de Emergencia	Intersección geométrica fuego-ventanas	0,35 ms	Negligible
Renderizado HUD	Dibujo de cajas, máscaras y texto táctico	0,85 ms	Negligible
Compresión Web	Codificación JPEG en memoria RAM	1,75 ms	Negligible
Difusión en Red	Transmisión asíncrona FastAPI (MJPEG/WS)	0,50 ms	Negligible
Total Estación Terrena	**Recepción UDP → Salida Web LAN**	**7.90 ms**	**2 650 MB VRAM (22.1 %)**

Tabla 3: Desglose de latencia media medida por fotograma en la estación terrena con YOLOX-26.

- **FPS Efectivos ($>$ 100 FPS):** Gracias a la arquitectura desacoplada y libre de anclas (*anchor-free*) de **YOLOX-26** y la aceleración FP16 en la GPU NVIDIA RTX 5070, el pipeline de visión artificial y retransmisión web procesa de manera sostenida a más de **108 FPS** (con un tiempo de ciclo total de apenas 7,90 ms por cuadro), superando con holgura los 100 FPS y triplicando la tasa de adquisición de la cámara del dron (30 FPS) para garantizar cero acumulación de búferes o retraso perceptual.
- **Latencia en Red Local:** La visualización en celulares y tablets conectados por Wi-Fi presenta un retraso menor a 8 ms adicionales respecto a la pantalla física de la estación terrena.
- **Concurrencia:** El servidor soporta múltiples dispositivos móviles concurrentes consumiendo menos del 15 % de CPU y sin degradación de la tasa de cuadros.